IILM UNIVERSITY

Greater Noida

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

C++ Presentation

OOPS WITH C++

VEHICLE TRACK

Vehicle Management System

A vehicle management system handling multiple vehicle types including electric vehicles.

Project Title

Vehicle Track

Course

Object-Oriented Programming with C++

Team Members

- Shelly Poonia (25SCS1003002956)
- Ayush Vishwakarma (25SCS1003003026)
- Naman (25SCS1003004379)

Section

B.Tech 1CSE 21

Problem Statement

Description of the Problem

The **Vehicle Management System** is an interactive, console-based C++ application designed to manage a diverse inventory of automobiles. This system is specifically built to handle and differentiate between traditional gasoline-powered cars and modern electric vehicles.

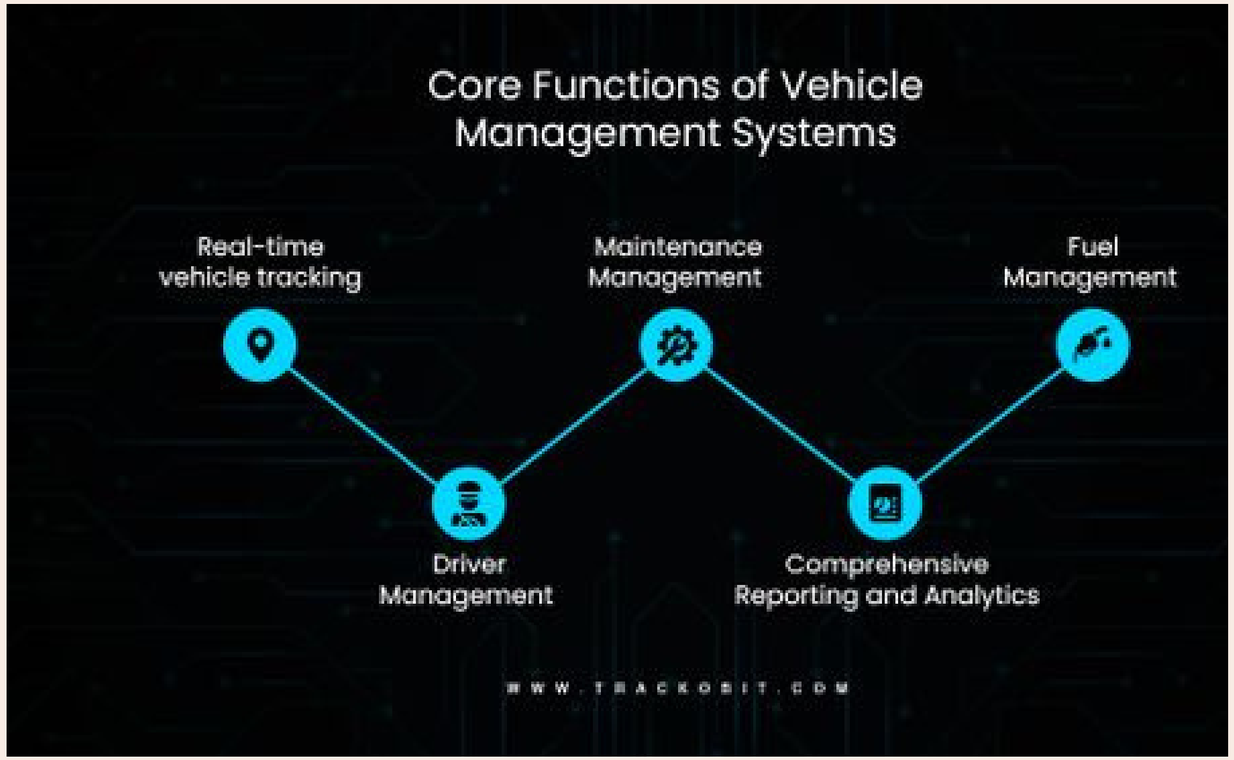
Real-World Scenario

This system can be used in car rental services to manage different types of vehicles like gas cars and electric vehicles.

Scope

This C++ application simulates a console-based inventory system that successfully differentiates, stores, and displays both gas and electric vehicles using a centralized manager.

Real-World Mapping



Usage of this project

Corporate Fleet Management: Tracking company cars for field researchers or executives.

Car Rental Agencies: Managing inventory where customers specifically request EVs or Gas cars.

Logistics & Delivery: Keeping track of vehicle range capabilities based on power source.

Who Are the Users?

- **Fleet Managers:** To input new vehicle purchases into the system.
- **System Administrators:** To view unified inventory reports at the end of the month.

```cpp
#include <iostream>
using namespace std;

class Vehicle {
protected:
    string make, model;
    int year;

public:
    Vehicle(string mk, string md, int yr) {
        make = mk;
        model = md;
        year = yr;
    }

    virtual void display() = 0;
};

class GasCar : public Vehicle {
    int fuel;

public:
    GasCar(string mk, string md, int yr, int f)
        : Vehicle(mk, md, yr), fuel(f) {}

    void display() {
        cout << "\n[Gas Car]\n"
             << make << " " << model << " (" << year << ")\n"
             << "Fuel: " << fuel << " Liters\n";
    }
};

class ElectricVehicle : public Vehicle {
    int battery;

public:
    ElectricVehicle(string mk, string md, int yr, int b)
        : Vehicle(mk, md, yr), battery(b) {}

    void display() {
        cout << "\n[Electric Vehicle]\n"
             << make << " " << model << " (" << year << ")\n"
             << "Battery: " << battery << " kWh\n";
    }
};

int main() {
    GasCar g("Honda", "City", 2020, 45);
    ElectricVehicle e("Tesla", "Model 3", 2022, 75);

    g.display();
    e.display();

    return 0;
}
```

KEY CODE SNIPPET

Output Demonstration

```
[Gas Car]
Honda City (2020)
Fuel: 45 Liters

[Electric Vehicle]
Tesla Model 3 (2022)
Battery: 75 kWh
namanchona@Namans-MacBook-Pro output %
```

Successful Execution: The program runs successfully and displays details of different vehicles.
When a gas car is added, it shows fuel capacity in liters, and for electric vehicles, it shows battery capacity in kWh.

Use of Functions: Each class has its own display() function which is used to print the details of that particular vehicle type in a clear format.

Correct Data Encapsulation: Each vehicle type stores its own data properly. Gas cars show fuel capacity in liters, while electric vehicles show battery capacity in kWh.

OOPS Concepts Used

OOPS Concepts & Architecture

Inheritance

A parent Vehicle class holds common data (Make, Model, Year). Child classes (GasCar and ElectricVehicle) inherit this data while adding their own specific attributes.

Encapsulation

Sensitive data like fuelCapacity and batteryCapacity are kept private and can only be accessed through the defined class methods.

Abstraction

The display() function hides internal implementation and shows only required details to the user. The user does not need to know how the data is processed internally.

THANK YOU